DEEP LEARNING · PROYECTO INTEGRADOR · 2026

VeritasIA: agente detector de noticias falsas y de su propagación

Plan de ejecución en 2 semanas. El agente recibe una noticia, estima la probabilidad de que sea verdadera o falsa y, si es probablemente falsa, pronostica cómo se propagaría en redes sociales durante las siguientes 48 horas. Sin RAG y sin imágenes.

Curso: Deep Learning
Equipo: 5 integrantes
Integrantes: _____
Catedrático: _____
Duración: 2 semanas (10 días hábiles)

1. Descripción del proyecto

VeritasIA es un agente que analiza una noticia escrita por el usuario. Primero, un modelo Transformer entrenado por el equipo calcula la probabilidad de que la noticia sea **verdadera** o **falsa**. Si la probabilidad de ser falsa supera un umbral, un segundo modelo (LSTM y Transformer para series de tiempo) **pronostica la curva de propagación**: cuántas veces se compartiría a lo largo de 48 horas, cuándo alcanza su pico y su alcance total estimado. Al final entrega un informe con ambos resultados.

1.1 Problema y justificación

La desinformación circula más rápido que su corrección, sobre todo en temporadas electorales, emergencias y crisis de salud. Un estudio publicado en *Science* en 2018 (Vosoughi, Roy y Aral) encontró que en Twitter las noticias falsas se difundían más lejos y más rápido que las verdaderas. Detectarlas temprano y estimar su potencial de difusión ayuda a medios, verificadores y autoridades a decidir qué desmentir primero.

1.2 Flujo del agente

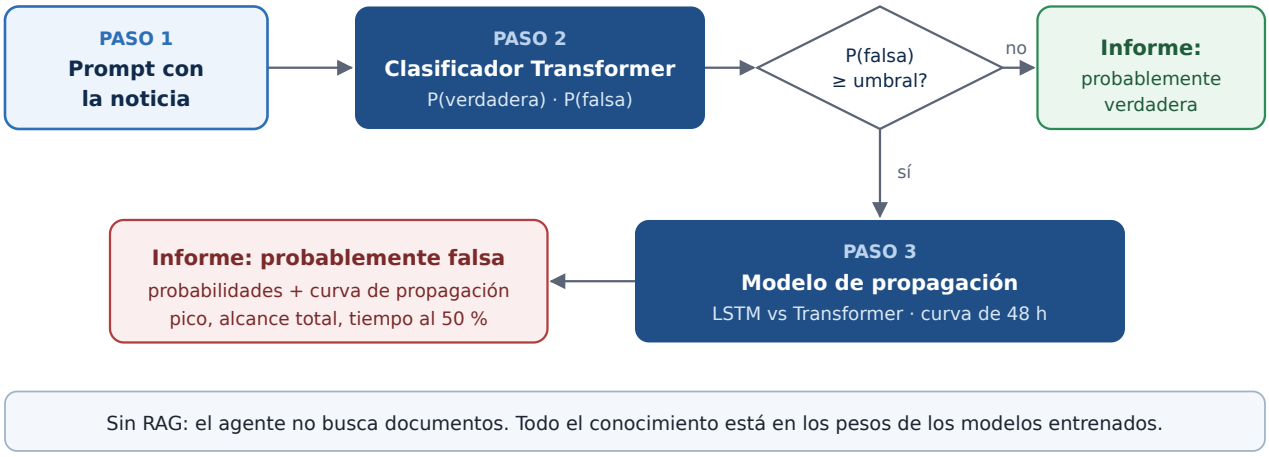


Figura 1. Flujo de 3 pasos. El modelo de propagación solo se ejecuta cuando la noticia es probablemente falsa.

1.3 Ejemplo de salida

Usuario: "URGENTE: el gobierno anunció que mañana cortarán el agua en toda la ciudad durante 15 días, compartan antes de que lo borren"

```
→ clasificar(texto) → P(verdadera) = 0.12 · P(falsa) = 0.88
→ P(falsa) ≥ 0.5 → propagar(texto) → curva de 96 pasos de 30 minutos
```

Agente: La noticia es **probablemente falsa (88 %)**. Se estima que alcanzaría su pico de difusión entre las horas 3 y 5 y que la mitad de su alcance total ocurriría en las primeras 8 horas. [Gráfica de la curva con banda de incertidumbre]

Los valores del ejemplo son ilustrativos.

2. Qué se aplica y qué no

2.1 Temas del curso

Sesión	Tema	¿Se aplica?	Dónde y cómo
S1	Introducción a DL	**Sí**	MLP sobre TF-IDF como línea base del clasificador; MLP como línea base de propagación
S2	CNN	**Sí**	CNN 1D para texto (filtros de 3, 4 y 5 palabras) como modelo de comparación del clasificador
S3	RNN, LSTM y GRU	**Sí**	BiLSTM en el clasificador; **LSTM para modelar la propagación en el tiempo** (núcleo del paso 3)
S4	Encoder-Decoder y Autoencoders	**Sí (parcial)**	Modelo encoder-decoder de propagación: el encoder lee la noticia y el decoder genera la curva paso a paso. Autoencoders no se usan
S5	Attention	**Sí**	Atención del decoder de propagación sobre las palabras de la noticia; visualización de qué palabras pesan más en la decisión del clasificador
S6	Transformers	**Sí**	**Clasificador principal** (BETO / XLM-RoBERTa) y Transformer para series de tiempo en propagación
S7	GAN	**No**	No aporta al flujo definido. Se justifica su exclusión en el informe
S8	Pérdidas, regularización y optimización	**Sí**	Cross-entropy ponderada, dropout, AdamW, warmup + cosine, early stopping y **calibración de probabilidades** (temperature scaling)
S9	Transfer Learning y Fine-tuning	**Sí**	Fine-tuning del Transformer preentrenado; transferencia entre idiomas (inglés → español)
S12	Series de tiempo y Forecasting	**Sí**	**Núcleo del paso 3:** pronóstico de la curva de propagación frente a baselines (curva promedio y modelo epidemiológico SIR)
S13	NLP	**Sí**	Tokenización, embeddings, BERT, limpieza del prompt y redacción del informe final
S14	IA Generativa	**Opcional**	Solo si sobra tiempo: VAE condicional para generar varios escenarios de propagación y una banda de incertidumbre

2.2 Lo que SÍ incluye

- Clasificación binaria verdadera / falsa con probabilidades calibradas.
- Comparación de 4 clasificadores: MLP, CNN 1D, BiLSTM y Transformer.
- Pronóstico de la curva de propagación a 48 horas para noticias falsas.
- Comparación LSTM vs. Transformer vs. baselines en propagación.
- Explicación de la decisión con mapas de atención.
- Agente que decide qué modelo ejecutar e interfaz web.

2.3 Lo que NO incluye

- **RAG:** no se buscan documentos ni bases de conocimiento.
- **Búsqueda en internet** ni consulta a verificadores externos.
- **Imágenes** ni videos: solo texto.
- **GAN** (S7).
- **Un LLM que decida** si es falsa: la decisión es del clasificador entrenado por el equipo.
- **Datos en tiempo real** de redes sociales: la propagación se pronostica, no se mide.

Limitación que debe quedar clara en el informe: sin RAG ni internet, el modelo no comprueba los hechos. Detecta **patrones de lenguaje y estilo** típicos de la desinformación (urgencia, alarmismo, falta de fuente, llamados a compartir). Por eso entrega probabilidades y no un veredicto definitivo, y no debe usarse para censurar contenido.

3. Datos

Conjunto	Contenido	Idioma	Uso
Spanish Fake News Corpus (Posadas-Durán et al., 2019) + conjunto de prueba de FakeDeS (IberLEF 2021)	Aprox. 970 noticias etiquetadas como verdaderas o falsas, más un conjunto de prueba adicional	Español	Clasificador (español)
LIAR (Wang, 2017)	Aprox. 12,800 afirmaciones cortas de PolitiFact en 6 niveles de veracidad; se agrupan en verdadera / falsa	Inglés	Clasificador (preentrenamiento)
Twitter15 y Twitter16 (Ma et al., 2017)	Aprox. 1,490 y 818 rumores con su tuit original, etiqueta (falso, verdadero, no verificado, no rumor) y **árbol de propagación con el tiempo de cada retuit**	Inglés	Propagación y clasificador
PHEME	Hilos de Twitter de 9 eventos noticiosos, con respuestas, marcas de tiempo y etiqueta de rumor / veracidad	Inglés	Propagación (validación)
Conjunto de prueba propio	50 noticias recientes en español (25 verificadas como falsas por medios de verificación, 25 verdaderas de medios formales), redactadas por el equipo	Español	Evaluación final, nunca para entrenar

3.1 Cómo construir las series de propagación

En Twitter15/16, cada línea del árbol indica quién compartió el tuit y **cuántos minutos después** de la publicación original. Con esos tiempos se construye, para cada noticia, una serie de 96 pasos de 30 minutos (48 horas) con el número acumulado de veces compartida, en escala logarítmica.

```
import ast, numpy as np

def curva_propagacion(ruta_arbol, horas=48, paso_min=30):
    tiempos = []
    for linea in open(ruta_arbol):
        _, hijo = linea.strip().split("->")
        uid, tweet_id, t = ast.literal_eval(hijo)      # t = minutos desde la publicación
        tiempos.append(float(t))
    bordes = np.arange(0, horas * 60 + paso_min, paso_min)
    conteo, _ = np.histogram(tiempos, bins=bordes)
    return np.log1p(np.cumsum(conteo))                 # alcance acumulado, 96 valores
```

3.2 Idioma

La mayoría de los datos de propagación están en inglés y los de clasificación en español son pocos. Usen un Transformer **multilingüe** (XLM-RoBERTa) o entrenen en inglés y ajusten luego

en español (S9). Comparar ambos enfoques es un buen experimento. Los usuarios pueden escribir en español y el modelo de propagación recibe la representación multilingüe del texto.

4. Modelos por paso

4.1 Paso 1: preparación del prompt

Limpieza del texto (URLs, emojis, espacios), detección de idioma y extracción de la noticia si el usuario escribe algo como "¿es falsa esta noticia?: ...". Se hace con reglas y expresiones regulares; no requiere un modelo.

4.2 Paso 2: clasificador verdadera / falsa

Modelo	Detalle	Temas
MLP	TF-IDF (5,000 términos) → 2 capas densas con dropout	S1
CNN 1D	Embeddings → filtros de 3, 4 y 5 palabras → max-pooling → capa densa	S2
BiLSTM	Embeddings → BiLSTM de 2 capas → capa densa	S3
Transformer (final)	XLM-RoBERTa base o BETO (`dccuchile/bert-base-spanish-wwm-cased`) con fine-tuning	S5, S6, S9

Probabilidades confiables (S8). Un clasificador puede decir "99 % falsa" y equivocarse con frecuencia. Para que las probabilidades signifiquen algo se aplica **temperature scaling**: se aprende un solo número T con el conjunto de validación y se divide la salida del modelo entre T.

```
T = torch.nn.Parameter(torch.ones(1))
opt = torch.optim.LBFGS([T], lr=0.01, max_iter=50)
def paso():
    opt.zero_grad()
    perdida = F.cross_entropy(logits_val / T, y_val); perdida.backward()
    return perdida
opt.step(paso)

probs = F.softmax(logits / T, dim=-1)     # [P(verdadera), P(falsa)]
```

4.3 Paso 3: modelo de propagación

Se entrenan con las curvas de Twitter15/16 y se comparan en dos modos:

- **Modo A, solo texto** (el que usa el agente): a partir de la noticia, generar la curva completa de 48 horas.
- **Modo B, pronóstico** (evaluación de S12): ver las primeras 6 horas reales y pronosticar las 42 siguientes.

Modelo	Detalle	Temas
Baseline 1: curva promedio	Curva promedio de las noticias falsas del entrenamiento	S12

Baseline 2: modelo SIR	Modelo epidemiológico clásico (susceptibles, infectados, recuperados) ajustado a los datos	S12
MLP	Embedding de la noticia → 96 valores	S1
Encoder-decoder LSTM con atención	El embedding inicializa el estado del LSTM; en cada paso, el decoder atiende a las palabras de la noticia y predice cuánto crece la curva	S3, S4, S5
Transformer para series	Decoder Transformer sobre los pasos de tiempo, con atención cruzada al texto; en modo B, también PatchTST	S6, S12

```
class Propagador(nn.Module):
    """Noticia (embeddings del Transformer) -> curva de alcance de 96 pasos."""
    def __init__(self, d_txt=768, h=256, pasos=96):
        super().__init__()
        self.pasos = pasos
        self.h0, self.c0 = nn.Linear(d_txt, h), nn.Linear(d_txt, h)
        self.lstm = nn.LSTMCell(1, h)
        self.mem = nn.Linear(d_txt, h)
        self.att = nn.MultiheadAttention(h, num_heads=4, batch_first=True)
        self.sal = nn.Linear(2 * h, 1)

    def forward(self, cls, tokens, y_real=None):          # y_real: teacher forcing
        h, c = torch.tanh(self.h0(cls)), torch.tanh(self.c0(cls))
        mem = self.mem(tokens)                             # (B, T, h)
        acum = torch.zeros(cls.size(0), 1, device=cls.device); curva = []
        for t in range(self.pasos):
            entrada = acum if y_real is None or t == 0 else y_real[:, t-1:t]
            h, c = self.lstm(entrada, (h, c))
            ctx, _ = self.att(h.unsqueeze(1), mem, mem)
            incremento = F.softplus(self.sal(torch.cat([h, ctx.squeeze(1)], -1)))
            acum = (acum if y_real is None or t == 0 else y_real[:, t-1:t]) + incremento
            curva.append(acum)
        return torch.cat(curva, dim=1)                     # curva no decreciente
```

El modelo predice incrementos positivos (softplus) y los acumula; así la curva nunca disminuye, igual que el alcance real.

4.4 El agente

```
def agente(prompt, umbral=0.5):
    texto = extraer_noticia(prompt)                 # paso 1: limpieza
    p_verdadera, p_falsa = clasificar(texto)        # paso 2: Transformer calibrado
    informe = {"P(verdadera)": p_verdadera, "P(falsa)": p_falsa}
    if p_falsa >= umbral:                           # paso 3 solo si es probablemente falsa
        curva = propagar(texto)                     # 96 pasos de 30 min
        informe |= {"pico_h": hora_pico(curva), "alcance": np.expm1(curva[-1]),
                    "t50_h": hora_mitad_alcance(curva), "curva": curva}
    return redactar(informe)                        # plantilla de texto + gráfica
```

5. Evaluación

Componente	Métricas	Comparación principal
Clasificador	Accuracy, F1 macro, AUC-ROC, matriz de confusión	MLP vs. CNN 1D vs. BiLSTM vs. Transformer
Probabilidades	ECE (error de calibración), Brier score, diagrama de confiabilidad	Antes vs. después de temperature scaling
Propagación modo A	RMSE y MAE de la curva (escala log), error en hora del pico, error en alcance final	Curva promedio vs. MLP vs. LSTM vs. Transformer
Propagación modo B	RMSE, MAE y MAPE de las 42 horas pronosticadas	SIR vs. LSTM vs. Transformer / PatchTST
Agente completo	Aciertos y tiempo de respuesta en las 50 noticias propias	Desempeño en español vs. inglés

Ablaciones (S8): con y sin pesos de clase, con y sin dropout, learning rate fijo vs. warmup + cosine. **Análisis adicional:** comparar las curvas promedio de noticias falsas y verdaderas para ver si se reproduce la diferencia reportada en la literatura.

6. Roles

Integrante	Rol	Responsable de	Temas
Integrante 1	Datos de texto y modelos base	Unificar conjuntos de texto, MLP, CNN 1D y BiLSTM	S1, S2, S3
Integrante 2	Clasificador Transformer	Fine-tuning, calibración, mapas de atención, ablaciones	S5, S6, S8, S9
Integrante 3	Datos de propagación y baselines	Árboles → series de tiempo, curva promedio, SIR, LSTM en modo B	S3, S12
Integrante 4	Modelos de propagación	Encoder-decoder LSTM con atención y Transformer (modo A)	S4, S5, S6, S12
Integrante 5	Agente e integración	Paso 1, lógica del agente, interfaz Gradio, informe, evaluación final, coordinación	S8, S13

7. Cronograma: 2 semanas

Día	Actividades	Al final del día
Semana 1: datos y modelos		
1	**Todos:** repositorio, entorno en Colab/Kaggle y entradas/salidas de cada función. I1 descarga y unifica conjuntos de texto. I3 descarga Twitter15/16 y PHEME. I5 arma las 50 noticias de prueba.	Datos descargados; funciones definidas
2	I1: limpieza y particiones (entrenamiento / validación / prueba). I3: árboles → curvas de 96 pasos. I2: primer fine-tuning del Transformer. I4: estructura del modelo de propagación. I5: paso 1 (limpieza del prompt).	**Datos listos y revisados**
3	I1: MLP y CNN 1D. I2: comparar XLM-R vs. BETO. I3: curva promedio y SIR. I4: encoder-decoder LSTM. I5: agente con funciones simuladas.	Primeros resultados
4	I1: BiLSTM. I2: temperature scaling. I3: LSTM en modo B. I4: atención en el decoder. I5: interfaz Gradio básica.	Agente con interfaz de prueba
5	I2: mapas de atención. I3 + I4: Transformer para series. I5: conectar modelos reales. **Revisión de equipo (1 h):** qué funciona y qué se recorta.	**Versión 0 de punta a punta**
Semana 2: integración, evaluación y entrega		
6	I1 + I2: ablaciones de S8. I3 + I4: PatchTST en modo B y ajuste de hiperparámetros. I5: gráfica de la curva y diseño del informe en la interfaz.	Modelos finales elegidos
7	Todos: corregir errores de integración; guardar modelos para que la demo no reentrene. (Opcional: VAE de escenarios.)	Demo estable
8	Evaluación final: métricas de todos los modelos, 50 noticias propias, análisis falsas vs. verdaderas. Tablas y gráficas.	Resultados finales
9	Informe técnico: cada integrante escribe su sección; I5 integra introducción, arquitectura, limitaciones, ética y conclusiones.	Informe completo
10	Presentación, ensayo de la demo con noticias preparadas, video de respaldo y entrega del repositorio.	**Entrega final**

8. Entregables y riesgos

- Repositorio en GitHub con carpetas `datos/`, `clasificador/`, `propagacion/`, `agente/`, `app/` y README reproducible.
- Aplicación Gradio: cuadro de texto, barra de probabilidades y gráfica de la curva de propagación.
- Informe técnico tipo artículo y presentación con demostración en vivo y video de respaldo.

Riesgo	Mitigación
Pocos datos en español	Transformer multilingüe y transferencia desde los datos en inglés; reportar el desempeño por idioma
El modelo aprende el estilo del medio y no la veracidad	Quitar nombres de medios, URLs y firmas antes de entrenar; probar con noticias de fuentes nuevas
Curvas muy variables (pocas noticias se vuelven virales)	Escala logarítmica, pérdida Huber y reportar errores por rangos de alcance
Los datos de propagación son de Twitter de hace varios años	Indicarlo como limitación; la curva es una estimación del patrón, no una predicción exacta
Probabilidades engañosas	Calibración con temperature scaling y mostrar siempre la incertidumbre

El primer día confirmen con el catedrático: (1) que acepta usar un Transformer preentrenado con fine-tuning como clasificador final, y (2) que la propagación se modele con datos públicos de Twitter, ya que no es posible obtener datos en tiempo real.